Zárthelyi dolgozat

Bevezetés

A mellékelt projektet egészítse ki/javítsa ki a megadott instrukciók alapján. A dependency injection során, ahol lehetséges az interface-re hivatkozzon. A projekthez mellékelten megtalálhtó egy Docker compose fájl, mely tartalmazza az adatbázis létrehozását. A függőségek között megtalálható a Liquibase, mely az adatbázis struktúrát kialakítja az alkalmazás elindításakor. Ennek a módosítására, kiegészítésére nincs szükség. Az entitások mezői definiálva vannak, de ezt a választott megoldás függvényében lehetséges, hogy annotációkkal és metódusokkal bővíteni kell, illetve olyan is lehet, hogy az adott technológiához további entitásokat szükséges felvenni. Szintén a választott technológiának megfelelően további függőségeket szükséges lehet definiálnia a projekthez!

Feladatok

1. CarDealerRepository

Pontszám: 20% (lépésenként 5-5%)

Készítsen egy CarDealerRepository nevű repository-t egy általa választott technológiával. (Spring JDBC, Spring Data JDBC, JPA, Spring Data JPA, stb.) A repository CarDealer entitásokat kezeljen és a következő műveleteket valósítsa meg:

1. Összes CarDealer listázása
2. Egy CarDealer betöltése ID alapján
3. CarDealer mentése (létrehozás és frissítés lehet egyben és külön is, a választott technológia függvényében)
4. CarDealer törlése

2. CarDealerService

Mapping

Pontszám: 20%

Készítse el az entitások és a DTO osztályok közötti mapping szolgáltatásokat. Lehetőleg MapStruct segítségével valósítsa meg ezt a feladatot! (Ennek hiányában maximum 10%-ot ér ez a feladat.)

Implementáció

Pontszám: 35% (metódusonként 7%)

Az előző feladatokban megvalósított CarDealerRepository és mapping felhasználásával implementálja a CarDealerService metódusait. Figyeljen arra, hogy ez az osztály a nem fogadhat és

nem adhat vissza entitásokat, csak DTO osztályokat! Továbbá figyeljen arra is, hogy a kereskedéseknek vannak forgalmazott járművei, melyeket szintén kezelni szükséges.

A következő metódusokat kell implementálnia:

1. *public List<CarDealerDTO> listAllCarDealers()*
2. *public CarDealerDTO loadCarDealer(Long id)*
3. *public CarDealerDTO create(CarDealerSaveDTO dto)*
4. *public CarDealerDTO edit(Long id, CarDealerSaveDTO dto)*
5. *public void deleteCar(Long id, Long vehicleId)*

Egy már meglévő entitás szerkesztése és törlése esetén figyeljen arra, hogy a megadott azonosító alapján tényleg létezzen az elem. Ha nem létezik, akkor dobjon kivételt! A módosító műveleteknél (létrehozás és szerkesztés) kezelje a járműveket is, tehát figyeljen arra, hogy a kapott jármű listából történhetett törlés, lehet, hogy bekerül újonnan forgalmazott jármű és az is lehet, hogy egy meglévő jármű adataiban történt változás.

3. CarDealerController

Pontszám: 20% (metódusonként 4%)

Készítse el a CarDealerController osztályt. Az összes mapping a */api/car-dealer* útvonal alá kerüljön. A következő metódusokat kell megvalósítani:

Szignatúra	Útvonal	HTTP metódus
public List<CarDealerDTO> listAllCarDealers()		GET
public CarDealerDTO loadCarDealer(Long id)	/[ID]	GET
public CarDealerDTO create(CarDealerSaveDTO dto)		POST
public CarDealerDTO edit(Long id, CarDealerSaveDTO dto)	/[ID]	PUT
public void removeCar(Long id, Long vehicleId)	/[ID]/[VEHICLE_ID]	DELETE

4. Dokumentáció és validáció

Pontszám: 5%

A projektben megtalálható a szükséges OpenAPI függőség. Ennek segítségével írjon rövid szöveges leírást az egyes metódusokhoz és a controller osztályhoz is. Validálja a kapott adatokat, hogy üres mezőt ne tudjon menteni.

5. Listázó képernyő – bónusz feladat

Pontszám: +10%

Készítsen egy egyszerű táblázatos képernyőt tetszőleges UI technológiában, melyhez az adatot az alkalmazás szolgáltatja.